

Gradient Boosting: the Intuition

Contents

1. Overview	2
2. Boosting passes mistakes forward	3
3. The toy data: five students	3
4. Stage 1: the mean	3
5. Pseudo-residuals: the mistakes of the current model	4
6. Stage 2: a tree that predicts the mistakes	4
7. Adding the models together	5
8. The learning rate: small steps in the right direction	6
9. Stage 3: the residuals of the combined model	6
10. Predicting for a new student	7
11. Watching it fit a curve	7
12. Learning rate against number of trees	10
13. Gradient boosting compared with AdaBoost	10
14. Summary	11
15. Sources	11
16. Key terms	11

Where this fits: Pipeline map steps 8 (Model), 10 (Tune). See the [Course map](#).



- **Builds on:** Gradient descent ([Note 57](#)); Log loss (binary cross entropy) ([Note 73](#)); Sigmoid function ([Note 74](#)); Regression trees ([Note 99](#)); Boosting ([Note 119](#)).
- **Leads to:** XGBoost ([Note 123](#)); Gradient descent ([Note 1017](#)).
- **Compare with:** AdaBoost ([Note 118](#)).

1. Overview

Key point: Gradient boosting starts with the mean as its first guess. Then, again and again, it trains a small regression tree on the current mistakes (the residuals) and adds a small part of that tree to the model.

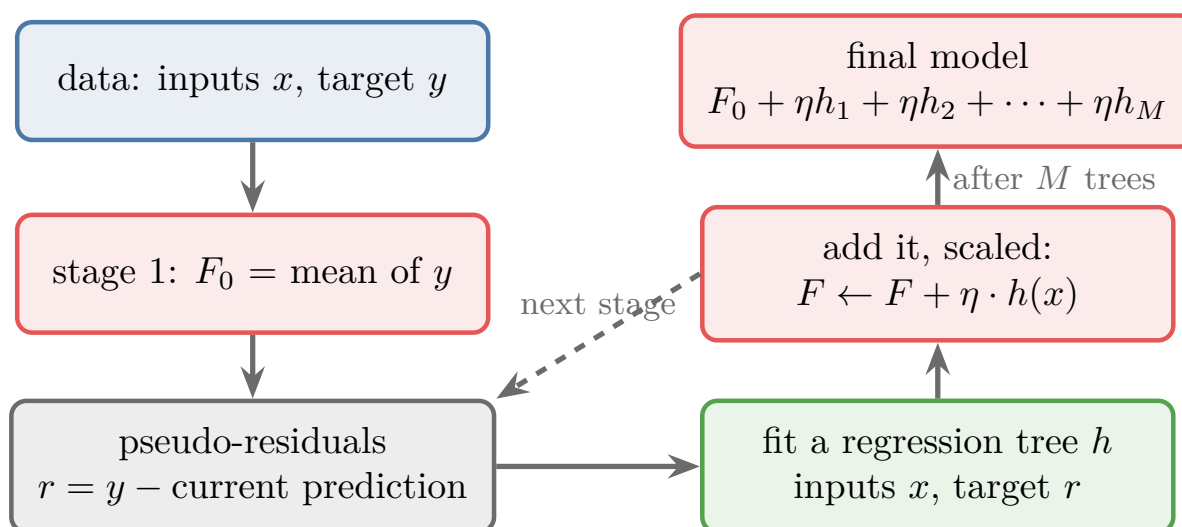


Figure 1: Gradient boosting for regression: a mean, then trees trained on residuals, each added with the learning rate

Gradient boosting is a boosting algorithm, like AdaBoost. It is one of the strongest algorithms for tables of data, and its optimised version, XGBoost, has won many Kaggle competitions: 17 of the 29 winning solutions on Kaggle's blog in 2015 used it (Chen and Guestrin 2016). Figure 1 shows the whole method for regression. This Note follows it by hand on five students, then watches it fit a curve. The maths behind each step is in the [gradient boosting maths Note](#); classification is in the [gradient boosting classification Note](#).

2. Boosting passes mistakes forward

Key point: Every boosting algorithm adds small models one at a time; each new model is told about the mistakes of the models before it. AdaBoost tells it through row weights; gradient boosting tells it through the residuals.

Boosting, from the [AdaBoost intuition Note](#), builds a big model from small models, added one after another: a stage-wise additive model. What makes it boosting is that each new model learns from the mistakes of the ones before.

The two algorithms pass on those mistakes in different ways:

- **AdaBoost** raises the weights of the rows it got wrong and draws a new dataset by weight, so the next stump focuses on those rows ([AdaBoost step by step Note](#)).
- **Gradient boosting** writes the mistakes down as numbers, one per row, and trains the next model to predict exactly those numbers.

3. The toy data: five students

Key point: Two inputs (IQ and CGPA), one numeric output (package in LPA): a regression problem.

Student	IQ	CGPA	LPA
1	90	8	3
2	100	7	4
3	110	6	8
4	120	9	6
5	80	5	3

LPA is lakh rupees per annum. We build an ensemble of three models: model 1, then two decision trees. Real ensembles use many more; three are enough to see every step.

4. Stage 1: the mean

Key point: For regression, the first model is not a tree at all: it is a single number, the mean of the output column, predicted for every row.

The first model ignores the inputs. Whatever the IQ and CGPA, it predicts the same number: the mean of the output column. In tree language it is a single leaf.

1. **In words:** add up the outputs and divide by the number of rows.
2. **Formula:**

$$F_0 = \frac{1}{n} \sum_{i=1}^n y_i$$

3. Example:

$$F_0 = \frac{3 + 4 + 8 + 6 + 3}{5} = \frac{24}{5} = 4.8$$

So model 1 says every student earns 4.8 LPA, even a student with CGPA 10 and IQ 120. We write F_0 for this first model and store its answers in a new column, *pred1*: 4.8 in every row. Why the mean is the right start is shown in the [gradient boosting maths Note](#), section 5.

5. Pseudo-residuals: the mistakes of the current model

Key point: The mistake on each row is actual minus predicted. Gradient boosting calls it the pseudo-residual.

To tell the next model about the mistakes, we need one number per row that says how wrong we were. We use actual minus predicted, called the **pseudo-residual**.

1. **In words:** for each row, subtract the current prediction from the true output.
2. **Formula:**

$$r_i = y_i - F_0$$

3. **Example:** student 1 earns 3 and was predicted 4.8:

$$r_1 = 3 - 4.8 = -1.8$$

Student	LPA	pred1	res1
1	3	4.8	-1.8
2	4	4.8	-0.8
3	8	4.8	3.2
4	6	4.8	1.2
5	3	4.8	-1.8

A negative residual means we predicted too much; a positive one, too little. A residual of 0 would mean a perfect prediction. The name “residual” comes from linear regression ([simple linear regression Note](#)); why gradient boosting adds “pseudo” is explained in the [gradient boosting maths Note](#), section 6.

6. Stage 2: a tree that predicts the mistakes

Key point: Model 2 is a regression tree with the same inputs, but its target is the residual column, not the package. It learns how wrong model 1 is for each kind of student.

This is the central idea of gradient boosting. Model 2 is a regression tree ([regression trees Note](#)):

- **inputs:** IQ and CGPA, as before;
- **target:** *res1*, the mistakes of model 1, not the package.

We are not asking tree 1 “what is this student’s package?” but “how far off is model 1 for this student?”. The better it predicts the mistakes, the better we can correct them.

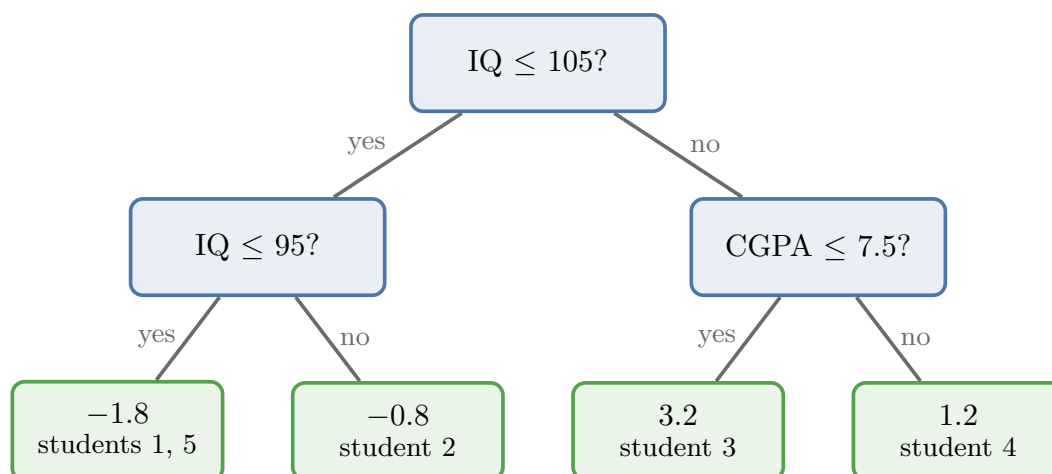


Figure 2: Tree 1, trained on the residuals: each leaf holds the residual of the students that reach it

Figure 2 shows the tree scikit-learn grows on this data. Student 1 (IQ 90) goes left at “IQ ≤ 105” and left again at “IQ ≤ 95”, reaching the leaf -1.8 : exactly student 1’s residual. With only five rows, every student gets a leaf of their own (students 1 and 5 share one, and their residuals are equal), so the tree’s predictions pred on the training rows equal the residuals exactly.

Extra: Any question that separates student 3 from student 4 gives the same right-hand leaves: “CGPA ≤ 7.5” and “IQ ≤ 115” both work, and scikit-learn happens to pick the second. Ties like this are common on tiny data and do not change the predictions on the training rows.

Trees are the usual choice for these later models: they give the best results with gradient boosting, although any regression model could be used.

7. Adding the models together

Key point: The ensemble’s prediction is model 1’s output plus model 2’s output. Without anything else, this reproduces the training data exactly, which is overfitting.

To predict, we add the outputs of the models.

1. **In words:** start from the mean and add the tree’s correction.
2. **Formula:**

$$F_1(x) = F_0 + h_1(x)$$

where h_1 is tree 1.

3. **Example:** student 1 (IQ 90, CGPA 8) gets 4.8 from model 1 and -1.8 from tree 1:

$$F_1 = 4.8 + (-1.8) = 3.0$$

which is exactly the package of 3 LPA.

Student 2 gets $4.8 - 0.8 = 4$, again exact. Every training row is now predicted perfectly, and that is a warning sign: the model has memorised these five students. A new student, not in the data, will probably be predicted badly. This is **overfitting** ([bias-variance Note](#)).

8. The learning rate: small steps in the right direction

Key point: We add only a fraction of each tree, the learning rate (typically 0.1). Each step is smaller, but it still points the right way; many small steps overfit less than one big jump.

The fix is to add only part of each tree's correction. That part is the **learning rate** η (eta), a number between 0 and 1, typically 0.1. With no learning rate, the multiplier is 1.

1. **In words:** multiply the tree's output by the learning rate before adding it.

2. **Formula:**

$$F_1(x) = F_0 + \eta h_1(x)$$

3. **Example:** student 1 with $\eta = 0.1$:

$$F_1 = 4.8 + 0.1 \times (-1.8) = 4.8 - 0.18 = 4.62$$

The answer, 4.62, is still far from 3, but it moved the right way. More trees will bring it closer, one small step at a time. The same idea, shrinking every model's contribution so that learning slows down and overfitting slows down even more, was called **shrinkage** in the [AdaBoost hyperparameters Note](#), section 4.

The learning rate is the same for every tree after the first model: with $\eta = 0.1$, tree 1, tree 2 and every later tree are all multiplied by 0.1.

9. Stage 3: the residuals of the combined model

Key point: The next tree is trained on the residuals of everything so far; with each stage the residuals move towards 0.

Before adding model 3, we measure how wrong the combined model of stage 2 is.

1. **In words:** actual minus the prediction of model 1 plus the scaled tree 1.

2. **Formula:**

$$r_i^{(2)} = y_i - (F_0 + \eta h_1(x_i))$$

3. **Example:** student 1:

$$r_1^{(2)} = 3 - (4.8 + 0.1 \times (-1.8)) = 3 - 4.62 = -1.62$$

Student	LPA	res1	pred2	res2	pred3	res3
1	3	-1.8	4.62	-1.62	4.458	-1.458
2	4	-0.8	4.72	-0.72	4.648	-0.648
3	8	3.2	5.12	2.88	5.408	2.592
4	6	1.2	4.92	1.08	5.028	0.972
5	3	-1.8	4.62	-1.62	4.458	-1.458

Every residual moved towards 0: -1.8 became -1.62 , 3.2 became 2.88 . A residual of 0 means no gap between actual and predicted, so the goal is to keep adding models until the residuals are close to 0.

Model 3 is again a regression tree: inputs IQ and CGPA, target $res2$. It predicts the mistakes of models 1 and 2 together. On this tiny dataset it splits like tree 1, with leaves -1.62 , -0.72 , 2.88 and 1.08 . Adding 0.1 times its output gives $pred3$, and $res3$ is smaller again.

10. Predicting for a new student

Key point: Send the new row through every tree, multiply each tree's output by the learning rate, and add everything to the mean.

1. **In words:** the mean, plus the learning rate times each tree's output.
2. **Formula:**

$$\hat{y} = F_0 + \eta h_1(x) + \eta h_2(x) + \cdots + \eta h_M(x)$$

3. **Example:** a new student with IQ 60 and CGPA 4.9 lands in the leaf " $\text{IQ} \leq 95$ " of both trees, with outputs -1.8 and -1.62 :

$$\hat{y} = 4.8 + 0.1 \times (-1.8) + 0.1 \times (-1.62) = 4.8 - 0.18 - 0.162 = 4.458$$

So the model predicts about 4.46 LPA. With five rows and two trees the answer is rough; with real data and hundreds of trees, the same procedure gives very good predictions.

11. Watching it fit a curve

Key point: On 100 points along a curve, the mean is a flat line; each tree bends the model towards the data, and after many trees the model starts chasing the noise.

The same steps on a bigger dataset show what each stage does. We take 100 points with one input x between -0.5 and 0.5 and the output $y = 3x^2$ plus a little random noise: a U-shaped, non-linear relationship. Each tree has at most 8 leaves.

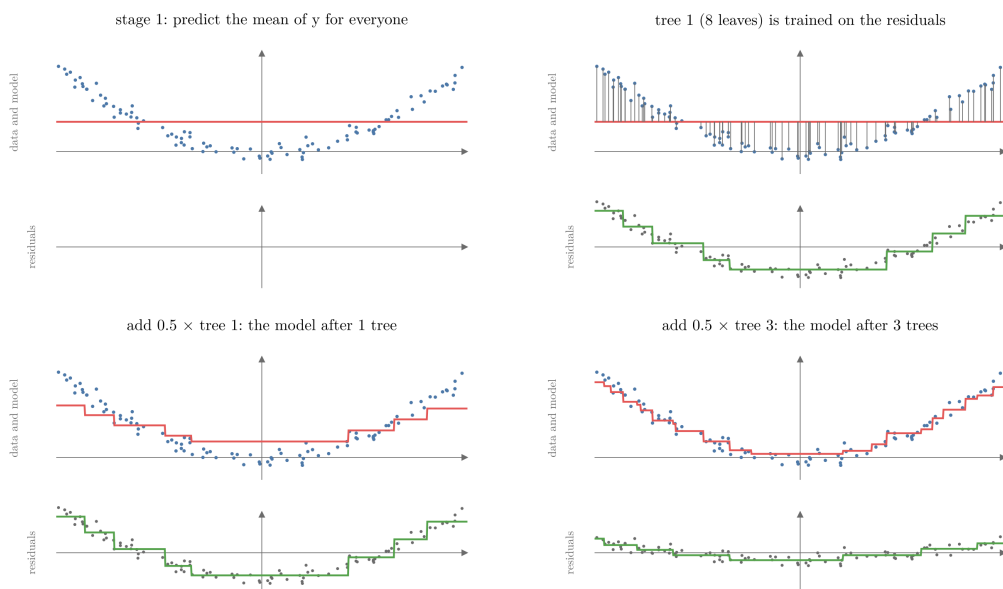


Figure 3: Gradient boosting on the curve, learning rate 0.5: residuals of the current model (grey), the tree trained on them (green), and the model after adding half of it (red)

Figure 3 runs three stages, with learning rate 0.5:

1. **Stage 1:** the model is the mean of y , 0.265: a flat red line.
2. **Residuals:** the grey sticks are the residuals; the lower panel shows them as points.
3. **Tree:** a tree with 8 leaves fits those residuals: the green steps.
4. **Update:** half of the green steps is added to the red line, which bends towards the data. The residuals shrink, and the next tree has less to fix.

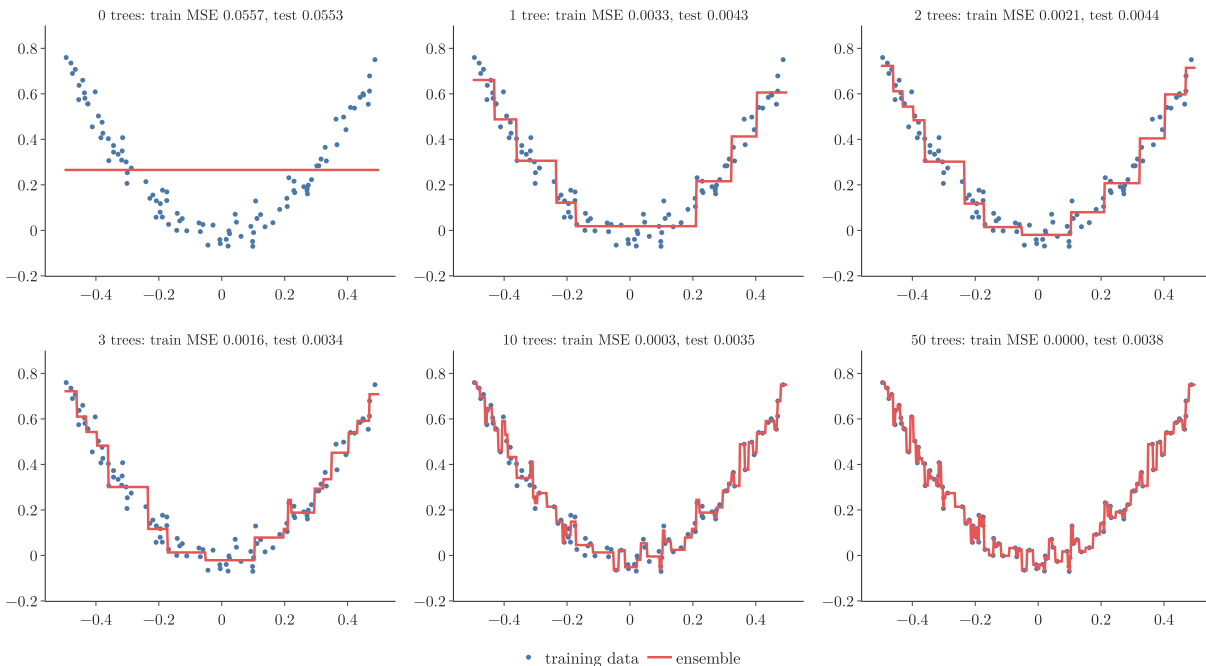


Figure 4: The model after 0, 1, 2, 3, 10 and 50 trees (learning rate 1, 8 leaves per tree), with training and test error

Figure 4 continues further, with learning rate 1. One tree already captures the U shape, and the training error falls from 0.0557 to 0.0033. After 10 and 50 trees the red line zigzags through every training point: the training error reaches 0, but the error on 100 new test points rises from 0.0034 (3 trees) to 0.0038. That is overfitting again, this time from too many trees.

Python: The whole loop is a few lines.

```
from sklearn.tree import DecisionTreeRegressor
```

```
pred = np.full(len(y), y.mean())          # stage 1: the mean
trees = []
for _ in range(n_trees):
    residual = y - pred                    # actual minus predicted
    tree = DecisionTreeRegressor(max_leaf_nodes=8)
    tree.fit(X, residual)                  # target: the residuals
    pred = pred + lr * tree.predict(X)     # small step
    trees.append(tree)
```

max_leaf_nodes=8 limits each tree to 8 leaves. scikit-learn's GradientBoostingRegressor(loss="squared_error", learning_rate=lr, max_leaf_nodes=8, max_depth=None) gives the same predictions to 15 decimals; the Notebook checks this. (max_depth=None matters: its default max_depth=3 would limit the trees as well.)

12. Learning rate against number of trees

Key point: As in AdaBoost, a smaller learning rate needs more trees but reaches a lower test error and overfits more slowly.

The trade-off is the one of the [AdaBoost hyperparameters Note](#), section 4.3, so the two are tuned together. On the curve, with 8 leaves per tree:

Learning rate	Lowest test error	Reached after	Test error after 200 trees
1.0	0.00316	6 trees	0.00378
0.5	0.00280	4 trees	0.00378
0.1	0.00259	33 trees	0.00349

With learning rate 1 the training error reaches 0 after about 50 trees, while the test error climbs back up. Learning rate 0.1 needs more trees, but its best test error is the lowest of the three.

The Notebook's playground (`app.py`, a Dash app) lets us change the number of trees, the learning rate and the leaves per tree, and watch the curve and both errors change.

13. Gradient boosting compared with AdaBoost

Key point: Gradient boosting uses bigger trees than AdaBoost's stumps, and one learning rate for every tree instead of a separate say for each model.

	AdaBoost	Gradient boosting
Base models	decision stumps (depth 1, 2 leaves)	trees with more leaves, commonly 8 to 32
How mistakes are passed on	larger weights for misclassified rows	the next tree's target is the residuals
How models are combined	each model gets its own say α	every tree is multiplied by the same learning rate
First model	a stump	the mean of the target (regression)

The leaf count depends on the data: about 8 leaves for small datasets, up to about 32 for big ones. Stumps are not used.

Extra: Sources differ on the best tree size. Trees with 4 to 8 leaves work well in most cases (ESL §10.11); scikit-learn's default, `max_depth=3`, allows at most 8. Like the learning rate, the tree size is a hyperparameter, best set by cross-validation.

14. Summary

Step	What we do	Toy example
Stage 1	predict the mean of y for everyone	4.8
Residuals	actual minus predicted	$-1.8, -0.8, 3.2, 1.2, -1.8$
Tree	regression tree, inputs x , target the residuals	leaves $-1.8, -0.8, 3.2, 1.2$
Update	add learning rate $\times$ tree	$4.8 + 0.1 \times (-1.8) = 4.62$
Repeat	new residuals, new tree	$-1.8 \rightarrow -1.62 \rightarrow -1.458$
Predict	mean $+ \eta \times$ (sum of tree outputs)	new student: 4.458

- Gradient boosting is a stage-wise additive model: each new tree predicts the mistakes of the ensemble so far.
- For regression the first model is the mean of the target, and the mistakes are the pseudo-residuals, actual minus predicted.
- Adding whole trees overfits; the learning rate adds only a fraction of each, and many small steps generalise better.
- Too many trees overfit as well, so the number of trees and the learning rate are tuned together.
- Unlike AdaBoost: bigger trees (commonly 8 to 32 leaves), and the same learning rate for every tree.

15. Sources

- Chen, T. and Guestrin, C. (2016). “XGBoost: A Scalable Tree Boosting System”. *Proceedings of KDD 2016*, Section 1.
- ESL: Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed., Springer. Section 10.11, “Right-sized trees for boosting”.

16. Key terms

Term	Meaning
Gradient boosting	A boosting algorithm that starts from a simple guess and adds trees one by one, each trained on the mistakes (pseudo-residuals) of the ensemble so far
Pseudo-residual	The mistake on one row that the next tree learns; for squared error it is actual minus predicted
Base prediction (F_0)	The first model of gradient boosting; for regression, the mean of the target

Term	Meaning
Learning rate (η) in gradient boosting	The fraction of each tree's output that is added to the model, the same for every tree; typically 0.1